

Voice → Internet → Speaking Avatar Assistant

A web app that hears a spoken question, fetches live internet data, and answers through a photorealistic talking avatar — with layered guardrails. | Stack: FastAPI · Vercel · Groq LLaMA-3.3-70B · Open-Meteo · edge-tts · Simli WebRTC

Q1 How to build a photorealistic, speaking, responsive human-like avatar IMPLEMENTED

A real-time talking avatar is a pipeline of four distinct AI/streaming stages. The naïve "render a video file then play it" approach is far too slow for conversation; the key is to **stream** so the face starts speaking on the first chunk of audio.

- 1. Face model (the likeness).** A photoreal human face is produced by a neural renderer — modern systems use diffusion- or NeRF-based models (and GAN variants) trained on a real person, so every frame is a genuine photorealistic image rather than a puppet mesh. This is what separates "human-like" from "cartoon."
- 2. Audio-driven lip-sync.** The answer *audio* drives mouth shapes (visemes) frame-by-frame, so lips match the phonemes being spoken. Getting this <100 ms and frame-accurate is the hard part and the reason to use a purpose-built API.
- 3. Expression / idle model.** Blinking, micro-head-motion and a "listening" idle between turns are what make it read as human rather than a frozen portrait.
- 4. WebRTC delivery.** Frames are streamed as a live **WebRTC** video track (not an MP4 download), giving sub-second glass-to-glass latency.

What I implemented. I use **Simli**, a real-time streaming-avatar API purpose-built for live agents (<300 ms, WebRTC, free tier). The browser:

```
on page load → pre-warm Simli session
(ICE → token → RTCPeerConnection
→ WebSocket SDP handshake)
answer text → edge-tts MP3
→ Web Audio API decodes to
16 kHz mono PCM
→ streamed over WebRTC datachannel
Simli face → lip-syncs & speaks live
in a <video> element
```

The session is **pre-warmed on page load** so no connection cost is paid at question time. A graceful **animated-SVG avatar** takes over if Simli is unavailable, so the demo never breaks.

Alternatives evaluated: HeyGen LiveAvatar and D-ID (hosted, similar quality, pricier trials); open-source SadTalker / Wav2Lip / MuseTalk (free but offline-speed on consumer GPUs → unsuitable for real-time). Simli won on latency + free tier.

Q2 What tools & models were used, and why

Stage	Choice	Why this one
Speech-to-text	Web Speech API (browser)	Zero cost, zero server latency, no key; text box fallback for unsupported browsers.
Backend	FastAPI on Vercel Python	Stateless request/response fits serverless; realtime channels stay browser-side (Vercel has no WebSockets). Python enables mocked pytest.
LLM (answer)	Groq llama-3.3-70b-versatile	Free tier with very high tokens/sec → lowest answer latency; OpenAI-compatible API. Prompt forces ≤2 spoken-style sentences, no markdown.
Guardrail L2	Same Groq model @ temp 0	Reuses one vendor for a safety classifier — defense-in-depth with no extra key or cost.
Live data	Open-Meteo (weather)	Keyless, reliable JSON geocoding + forecast; responses cached 5 min. (Tavily/Serper optional for general search.)
Text-to-speech	edge-tts (MS neural) + browser TTS	Free, natural voice; MP3 is decoded to PCM to drive avatar lip-sync while browser TTS plays the audible voice in parallel.
Avatar	Simli real-time WebRTC	Purpose-built for live agents, <300 ms, generous free tier, simple SDK — best real-time option in the time box.
Hosting	Vercel	Free, HTTPS (required for mic access), instant public demo URL; keys in env vars, never in client code.

Guardrails (a judged criterion — over-invested). Three fail-safe layers: **L1** regex/keyword blocklist (profanity, slurs, violence, self-harm, illegal acts + prompt-injection patterns like "ignore your instructions", "DAN", "[system]") — instant, no LLM wasted; **L2** Groq LLM classifier catches indirect/obfuscated asks regex misses, and **fails open** so a hiccup never blocks a real user; **L3** scans the final answer before it is spoken. Refusals are polite and spoken by the avatar. User text is never concatenated into the system prompt. Verified by a mocked pytest suite (**28 passing**, zero API quota) plus a live red-team run — **44/44** prompts handled correctly (26/26 unsafe blocked, 18/18 benign answered, zero false refusals).

Q3 How to reduce lag & latency in the video + response pipeline

Implemented:

- Pre-warm the WebRTC avatar session on page load** — the single biggest win; the connection cost is paid before the user speaks.
- Fastest model tier** — Groq LLaMA-3.3-70B for very high tokens/sec; answers capped at ≤2 sentences (less text → less audio → less video to generate).
- Parallel audio paths** — audible browser TTS and the avatar's PCM lip-sync run concurrently, not sequentially.
- Pre-fetch + cache the tool call** — weather is fetched and injected as context, and cached 5 min, so repeat queries skip the network hop.
- Interim STT** — transcript renders live as the user speaks, hiding STT latency.

Further (roadmap): stream LLM tokens via SSE and start speaking on the *first sentence*; edge deployment near the avatar provider's region; sentence-chunked pipelining; speculative pre-fetch of likely tool calls.

Representative end-to-end latency (p50) — per stage, question→avatar-speaking:

Stage	p50
STT (speech → text, in-browser)	~0.3 s
Guardrail L1 (regex)	<5 ms
Guardrail L2 (LLM classifier)	~0.4 s
Weather tool (cached / live)	0–0.3 s
LLM answer (Groq)	~0.6 s
TTS (edge-tts → PCM)	~0.4 s
Avatar first frame (pre-warmed)	~0.3 s
Total (question → speaking)	~3–4 s

Figures are representative p50 on the deployed Vercel build in Chrome; pre-warming removes ~1–2 s of one-time WebRTC setup from the hot path.